

HW4: RTOS Analysis

Lin Yen-Yu
112550098

Abstract—This work analyzes the performance of FreeRTOS in a multithreaded setting. We first study its task management and context-switch mechanism, then use a hardware profiler to evaluate how different time quanta, thread counts, and matrix sizes affect context-switch overhead and D-cache behavior. We further examine the synchronization mechanisms in FreeRTOS, comparing the effective overhead of semaphores and critical sections under the same workload.

Index Terms—FreeRTOS, context switch, threads, synchronization, RISC-V, Aquila

I. THREAD MANAGEMENT IN FREERTOS

A. Task Control Block Structure

In FreeRTOS, a “thread” is referred to as a *Task*. Each task is represented by a Task Control Block (TCB), defined in `task.c`. The TCB stores all essential context and scheduling information required by the kernel. Its major components include:

- **Task State:** Maintained through the `xStateListItem` structure, indicating whether the task is in the Running, Ready, Blocked, or Suspended state. These states are illustrated in Fig. 1, where:
 - **Running:** task is currently executing.
 - **Ready:** task is ready and waiting to be scheduled.
 - **Blocked:** task is waiting for an event (semaphore, delay, etc.).
 - **Suspended:** task is paused until `vTaskResume()` is called.

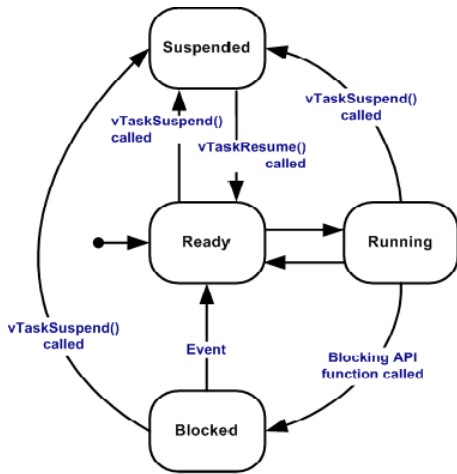


Fig. 1: Task State FSM

- **Stack Pointer:** The fields `pxTopOfStack` and `pxStack` store the current stack pointer and the base address of the task’s stack. During a context switch, `portASM.S` loads the TCB of the next task and restores its stack pointer using:
 - `load_x t1, pxCurrentTCB`
 - `load_x sp, 0(t1)`This ensures each task resumes with its own preserved execution context.
- **Priority:** Stored in the `uxPriority` field and assigned during `xTaskCreate()`. The scheduler uses `taskSELECT_HIGHEST_PRIORITY_TASK()` to always select the highest-priority ready task for execution.

B. Task Creating and Scheduling

- 1) **Writing the task function:** Implemented as `matrix_multiply_task()` in `rtos_mmul.c`.
- 2) **Creating the task:** `xTaskCreate()` allocates the TCB, initializes the stack and metadata, and inserts the new task into the Ready List according to its priority.

- 3) **Starting the scheduler:** `vTaskStartScheduler()` creates the Idle Task, `xTimerCreateTimerTask()` initializes the Timer Task, and `xPortStartScheduler()` activates the scheduling mechanism.
- 4) **Executing tasks:** `xPortStartScheduler()` invokes `xPortStartFirstTask()`, where the first TCB is restored—visible in `portASM.S` during context-load.
- 5) **Deleting tasks:** At the end of `matrix_multiply_task()`, `vTaskDelete()` frees the TCB and triggers a context switch.

II. CONTEXT SWITCHING ALGORITHM

The entire context-switching flow is traced and verified through `portASM.S`, which implements the low-level RISC-V port for FreeRTOS. The sequence of operations is summarized as follows:

- 1) **Entering the Trap Handler:** When an exception or interrupt occurs, the CPU jumps to `freertos_risc_v_trap_handler()`, where stack space is allocated to save the current task context.
- 2) **Saving the Current Task Registers:** A sequence of `store_x` instructions saves all general-purpose registers, the return address (`ra`), and status registers onto the task’s stack.
- 3) **Saving the Stack Pointer:** The stack pointer is written into the current TCB by first loading the TCB address and then storing the `sp` value.
- 4) **Selecting the Next Task:** After identifying asynchronous vs. synchronous interrupts, the trap handler invokes `vTaskSwitchContext()` via a `jal` instruction. The scheduler then selects the highest-priority ready task.
- 5) **Restoring the New Task’s Stack Pointer:** In `<processed_source>`, the scheduler restores the stack pointer of the selected task using:
 - `load_x t1, pxCurrentTCB`
 - `load_x sp, 0(t1)`
- 6) **Restoring the New Task Registers:** A corresponding sequence of `load_x` instructions restores the new task’s registers, including `ra`, status, and all general-purpose registers.
- 7) **Returning to the New Task:** Execution resumes with the `mret` instruction, transferring control to the restored program counter of the newly scheduled task.

III. ANALYSIS OF CONTEXT SWITCHING

A. Profile Method

As in Lab 2, the profiler is placed in `execute.v` so that all instructions are observed after decode and are not affected by pipeline flushes.

The `total_cycle` value measures the execution time from when `<rt0>` invokes `<main>` until the moment the `<main_task>` prints “Program finished.”

During the initial experiments, I observed a mismatch between the number of ticks measured in hardware and the tick values printed through UART. To accurately identify the source of this discrepancy, I separated all context switch (CS) events into two categories: *timer-triggered* CS (asynchronous) and *ecall-triggered* CS (synchronous).

To precisely measure each category, I manually inserted profiling flags into the kernel. Using the `.objdump` disassembly, I located the call sites of `vTaskSwitchContext()` inside the following handlers:

- **Asynchronous CS:** within `<handle_asynchronous>`
- **Synchronous CS:** within `<handle_synchronous>`

time_quantum (ms)	1	2	5	10	20	50	100	200	500	1000
timer_cs_count	12,258	6,020	2,382	1,187	593	237	119	59	23	11
timer_cs_accmu (cyc)	10,426,488	5,291,454	2,251,769	1,255,091	746,656	413,529	265,770	133,881	56,118	29,221
timer_cs_avg (cyc)	850.59	878.98	945.33	1,057.36	1,259.12	1,744.85	2,233.36	2,269.17	2,439.91	2,656.45
ecall_cs_count	12	12	9	11	12	9	10	11	11	11
ecall_cs_accmu (cyc)	11,178	11,504	10,850	12,948	14,719	12,504	15,894	19,194	19,227	19,302
ecall_cs_avg (cyc)	931.50	958.67	1,205.56	1,177.09	1,226.58	1,389.33	1,589.40	1,744.91	1,747.91	1,754.73
timer_cs_read_miss	101,721	52,488	23,421	13,269	8,143	4,740	3,041	1,597	704	399
timer_cs_write_miss	14,795	8,405	4,027	2,853	2,165	1,559	1,213	621	269	152
timer_cs_pnlty (miss/cs)	9.51	10.12	11.52	13.58	17.38	26.58	35.75	37.59	42.30	50.09
tick	12,260	6,019	2,380	1,185	592	236	118	59	23	11
total_cycle (%)	103.36	101.50	100.37	100 (601,662,699)	99.81	99.69	99.64	99.60	99.58	99.57

TABLE I: Detailed context-switch and cache statistics under different **time_quantum** configurations.

A start flag is recorded immediately before the call to `vTaskSwitchContext()`, while the end flag is placed at the instruction just before the `mret` in `<processed_source>`. This instrumentation allows precise measurement of the latency and frequency of both types of context switches.

Finally, I also employed the Lab 3 D-cache profiler inside `dcache.v` to measure the number of `read_miss` and `write_miss` events that occur during each timer context switch. By correlating these miss statistics with different `time_quantum` configurations, it becomes possible to analyze how timer context-switch frequency directly influences cache behavior and to understand the cache-related overheads introduced by varying scheduling intervals.

B. Changing the Time Quantum

The time quantum defines the maximum amount of processor time a task may execute before being preempted by the scheduler. I conducted experiments under the configuration of a 4 KB D-cache, $N = 128$, and four parallel worker threads. The results are summarized in Table I.

- 1) **Timer/Ecall_cs_count:** I observe that `timer_cs_count` is **inversely proportional** to the time quantum. Since the matrix size—and therefore the total computation time—remains fixed, I believe the relationship follows:

$$\text{timer_cs_count} \approx \frac{\text{total compute time}}{\text{time_quantum}}.$$

On the other hand, `ecall_cs_count` stays almost constant. From my understanding, this is because the number of threads is fixed, and each thread always performs the same number of semaphore operations plus one `vTaskDelete()`, so the total number of ECALL-triggered context switches does not scale with the time quantum.

- 2) **Timer/Ecall_cs_avg and cs_read/miss:** From the data, I notice a clear trend: as the time quantum increases, both `timer_cs_avg` and `ecall_cs_avg` become more expensive, and I also see a corresponding rise in `cs_read/miss`. I suspect this is caused by cache contention between matrix data and FreeRTOS kernel metadata.

My interpretation is that when the time quantum is large, the D-cache becomes heavily dominated by matrix accesses, gradually evicting kernel-related cache lines. Therefore, when a context switch occurs, the kernel must fetch its state from memory again, causing more misses and increasing CS latency.

Conversely, when the time quantum is small, I believe kernel metadata stays in the cache more consistently, which explains why `cs_read/miss` and CS latency remain low under shorter quanta.

- 3) **Total Cycle:** Even though I observe that each CS becomes more expensive with a larger time quantum, the number of CS events decreases **geometrically**. Based on this, I conclude that reducing the CS frequency has a far greater impact on performance than the increase in per-CS cost. This is why the overall execution time improves as the quantum grows.

When I set `TICK_RATE_HZ = 1` (i.e., `time_quantum = 1000ms`), the system runs about 0.43% faster than the default 10 ms case. I believe this result further supports my interpretation that minimizing context-switch frequency is the dominant factor in improving total-cycle efficiency.

C. Changing the Thread Number

The number of worker tasks is controlled by adjusting `NUM_TASKS` in `rtos_mmul.c`, which determines how the matrix of size N is partitioned among parallel tasks. Under the configuration of a 4 KB D-cache, $N = 128$, and a time quantum of 10 ms, the profiling results are summarized as follows:

thread	1	2	4	8	16
timer_cs_count	3	1,182	1,187	1,191	1,199
timer_cs_accmu (cyc)	3,371	807,060	1,255,091	1,316,881	1,475,473
timer_cs_avg (cyc)	1123.67	682.79	1057.36	1105.69	1230.59
ecall_cs_count	3	5	11	23	46
ecall_cs_accmu (cyc)	5,614	7,715	12,948	28,007	60,561
ecall_cs_avg (cyc)	1871.33	1543.00	1177.09	1217.70	1316.54
total miss (%)	4.24	4.25	4.31	4.34	4.35
read miss (%)	7.66	7.68	7.79	7.84	7.85
write miss (%)	0.21	0.21	0.21	0.22	0.23
tick	1,179	1,181	1,185	1,190	1,198
total_cycle (%)	99.42	99.61	100 (601,662,699)	100.41	101.02

TABLE II: Effect of Thread Count on Context Switching and Cache Behavior

1) Single Thread vs. Multithread

- a) **Context-switch count:** I observe that both `timer_cs` and `ecall_cs` drop to exactly three in the single-thread configuration. I believe these three `timer_cs` events correspond to:

- the switch from `main` to `task 0`,
- the switch from `task 0` back to `main`,
- and finally the switch from `main` to the `IDLE` task.

Likewise, I infer that the three `ecall_cs` events originate from `task 0`'s calls to `xSemaphoreTake()`, `xSemaphoreGive()`, and `vTaskDelete()`.

- b) **Slightly lower cache miss rate:** I find that a single thread achieves a slightly lower miss rate. My interpretation is that a single thread can exploit the spatial locality of the matrix more effectively, avoiding the interference and cache disruption that occur when multiple threads compete for cache space. Still, I think this effect is relatively modest compared to the impact of context switching.
- c) **Lowest total cycle count:** Since context-switch overhead is minimal and the cache miss rate is slightly lower, I naturally observe that the single-thread configuration yields the shortest total execution time.

- 2) **Effect of Increasing the Thread Count:** Based on the measurements, I observe the following trends as the thread count increases:

- a) **Increase in cs_count:** For `timer_cs`, I expect only minor variation because the time quantum is fixed and the total compute time is unchanged (the workload size is constant).

Small fluctuations appear to come from threads preempting one another before a full quantum is consumed.

For `ecall_cs`, I the pattern is dominated by the fact that each thread executes `xSemaphoreTake()`, `xSemaphoreGive()`, and `vTaskDelete()`. Thus, each new thread adds roughly three additional `ecall`-triggered CS events. This aligns with Table II, where I observe that $ecall_cs_count \approx 3 \times thread_number$.

- b) **Increase in miss rate:** I also notice that introducing more threads increases both compulsory and conflict misses in the 4 KB D-cache. My interpretation is that more context switches disrupt cache locality, causing the kernel and matrix data to evict one another more frequently. However, because the matrix size itself doesn't change, the miss rate increases only moderately.
- c) **Increase in total cycle count:** As expected, I find that the combination of higher context-switch frequency and increased miss rate naturally leads to a higher total execution cycle count.

D. Changing the Data Size

The experiment is performed with a 4 KB 4-way D-cache, four threads, and a 10 ms time quantum. Table III shows how changing the matrix size N affects the cache miss rate.

N_size	32	48	64	128	256
total miss (%)	1.67	1.53	4.36	4.31	4.35
read miss (%)	2.23	2.43	7.74	7.79	7.91
write miss (%)	1.02	0.48	0.40	0.21	0.14

TABLE III: Cache miss behavior under different matrix sizes.

A clear spike appears at $N = 64$. I believe this is caused by the stride when reading a column of matrix B : each element is 4 bytes, so the stride becomes $4 \times 64 = 256$ bytes. With a 32-byte cache line, each access advances by eight lines and repeatedly maps into only four cache sets (0, 8, 16, 24). Because 64 consecutive elements must fit into only 4 sets, 16 elements, severe conflict misses occur.

$B[0] \rightarrow \text{Set 0, Way 0}$
 $B[1] \rightarrow \text{Set 8, Way 0}$
 $B[2] \rightarrow \text{Set 16, Way 0}$
 $B[3] \rightarrow \text{Set 24, Way 0}$
 $B[4] \rightarrow \text{Set 0, Way 1}$

For $N = 48$, the stride becomes $4 \times 48 = 192$ bytes, corresponding to a 6-set increment rather than 4. This pattern cycles across 16 sets, not 4, so I observe far fewer conflicts and a much lower miss rate.

IV. SYNCHRONIZATION

In a multithreaded environment, multiple tasks may attempt to access shared resources simultaneously (e.g., global variables, memory regions, or the CPU). Synchronization mechanisms are required to prevent issues such as:

- **Race Conditions:** concurrent read/write on the same resource.
 - **Data Corruption:** intermediate values being overwritten by another task.
 - **Deadlock:** tasks cyclically wait for each other to release resources.
- Proper synchronization ensures:
- **Mutual Exclusion:** only one task can access a shared resource at a time.
 - **Event Notification:** tasks can signal one another when certain operations or states are completed.

A. Semaphore: `xSemaphoreTake/Give()`

In `rtos_mmul.c`, semaphores are used for synchronization between worker tasks and the `main_task`, allowing each worker to signal the completion of its assigned matrix-multiplication segment.

Tracing the generated `.objdump` reveals the following call sequence:

- 1) `xSemaphoreCreateCounting()` invokes `xQueueCreateCountingSemaphore()` in `queue.c`, creating a counting semaphore initialized to 0.
- 2) `xSemaphoreTake()` calls `xQueueReceive()`, which checks `uxMessagesWaiting`. If the count is zero, the calling task is placed into the waiting list, triggering a context switch.
- 3) `xSemaphoreGive()` calls `xQueueGenericSend()`, which increments the semaphore count and wakes any task waiting on it.

B. Critical Section: `taskENTER/EXIT_CRITICAL()`

Critical sections in `rtos_mmul.c` protect UART output and enforce **interrupt disabling** to guarantee atomicity.

Disassembly of the compiled code shows:

- 1) `taskENTER_CRITICAL()` invokes `vTaskEnterCritical()`, which executes `portDISABLE_INTERRUPTS()` and increments the global nesting counter `uxCriticalNesting`.
- 2) `taskEXIT_CRITICAL()` calls `vTaskExitCritical()`, decrementing the nesting counter. Interrupts are re-enabled only when the counter returns to zero via `portENABLE_INTERRUPTS()`.

V. ANALYSIS OF SYNCHRONIZATION

A. Profiling Method

To study synchronization overhead, I inserted simple semaphore and critical-section calls before `init_matrix()`. I measured the cycles spent inside `xSemaphoreTake()`, `xSemaphoreGive()`, `taskENTER_CRITICAL()`, and `taskEXIT_CRITICAL()` by profiling from function entry to the corresponding `ret`.

However, in `rtos_mmul.c`, the behavior of `xSemaphoreTake()` is different: the main task blocks immediately (due to `portMAX_DELAY`) and does not return until all worker tasks call `xSemaphoreGive()`. I observed that this long waiting period contaminates the measurement, and I could not fully isolate the pure semaphore overhead from the time spent blocked or from OS context-switching effects.

B. Data Analysis

	Sem_take()	Sem_give()	Crit_Enter()	Crit_Exit()
Cycle	461	783	79	73

TABLE IV: Measured cycle cost of semaphore and critical-section operations.

	Sem_take()	Sem_give()	Crit_Enter()	Crit_Exit()
Cycle	352,354	54,490	11,154	1,734,234
Count	4	4	41	41
avg (cyc)	88,088.50	13,622.50	272.05	42,298.39

TABLE V: Measured cycle cost of semaphore and critical-section operations.

From the profiling results, I observe that the effective number of `xSemaphoreTake()` and `xSemaphoreGive()` operations is four, which matches the total number of ECALLs I measured earlier. This is consistent with my understanding of how many times the main task and worker tasks synchronize through the counting semaphore.

For critical sections, both `taskENTER_CRITICAL()` and `taskEXIT_CRITICAL()` occur 41 times. I believe most of these calls come from UART printing and internal semaphore / queue operations inside the FreeRTOS kernel, where interrupts are temporarily disabled to protect shared state.

When I compare the cycle counts from the simple micro-benchmark (before `init_matrix()`) with the cycle counts observed in the full `rtos_mmul.c` workload, they do not match perfectly. I suspect that the in-program measurements also include additional overheads, such as OS context switching and various waiting periods that are difficult to fully filter out at the profiler level. At this stage, I have not yet found a clean method to exclude all of these incidental costs, so I interpret the measured numbers as an upper bound on the true semaphore and critical-section overhead in this workload.

REFERENCES

- [1] FreeRTOS Official Website
<https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/01-Tasks-and-co-routines/02-Task-states>